Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# --- 1. Loading the Dataset ---
# Set the file path to your training data
file_path = 'train.csv'
df = pd.read_csv(file_path)

print("--- Original Data Info ---")
#df.info()
#print("\nMissing values before cleaning:\n", df.isnull().sum())


# --- 2. Handling Missing Values ---
# For categorical columns, we fill missing values with the mode (most frequent value)
df['Gender'].fillna(df['Gender'].mode()[0], inplace=True)
df['Married'].fillna(df['Married'].mode()[0], inplace=True)
df['Dependents'].fillna(df['Dependents'].mode()[0], inplace=True)
df['Self_Employed'].fillna(df['Self_Employed'].mode()[0], inplace=True)

# For the 'Credit_History' column (which is categorical but numeric), mode is most appropriate
df['Credit_History'].fillna(df['Credit_History'].mode()[0], inplace=True)

# For numerical columns, I'll fill missing values with the median to be robust to outliers
df['LoanAmount'].fillna(df['LoanAmount'].median(), inplace=True)
df['Loan_Amount_Term'].fillna(df['Loan_Amount_Term'].median(), inplace=True)


# --- 3. Correcting Data Types and Values ---
# The 'Dependents' column has a '3+' value, which needs to be replaced with a number
df['Dependents'] = df['Dependents'].replace('3+', '3')
# Convert the column to a numeric type
df['Dependents'] = pd.to_numeric(df['Dependents'])


# --- 4. Feature Engineering ---
# Create a 'Total_Income' feature for a more holistic view of applicant's income
df['Total_Income'] = df['ApplicantIncome'] + df['CoapplicantIncome']

# Applying a log transformation to skewed numerical features to normalize their distribution
```

```
df['ApplicantIncome_Log'] = np.log1p(df['ApplicantIncome'])
df['CoapplicantIncome_Log'] = np.log1p(df['CoapplicantIncome'])
df['LoanAmount_Log'] = np.log1p(df['LoanAmount'])
df['Total_Income_Log'] = np.log1p(df['Total_Income'])


# --- 5. Encoding Categorical Variables ---
# Dropping the original skewed columns and non-predictive Loan_ID
df = df.drop(columns=['ApplicantIncome', 'CoapplicantIncome', 'LoanAmount', 'Total_Income',
'Loan_ID'])

# Convert binary categorical variables to 0s and 1s
df['Gender'] = df['Gender'].map({'Male': 1, 'Female': 0})
df['Married'] = df['Married'].map({'Yes': 1, 'No': 0})
df['Education'] = df['Education'].map({'Graduate': 1, 'Not Graduate': 0})
df['Self_Employed'] = df['Self_Employed'].map({'Yes': 1, 'No': 0})
df['Loan_Status'] = df['Loan_Status'].map({'Y': 1, 'N': 0})

# Use one-hot encoding for 'Property_Area' as it has more than two categories
df = pd.get_dummies(df, columns=['Property_Area'], drop_first=True)

#print("\n--- Cleaned Data Info ---")
#df.info()
#print("\nPreview of the final, cleaned data:")
#print(df.head())

# ============================================================================
# --- 6. Preparing Data for Modeling ---
# ============================================================================

# Separating features (X) and target variable (y)
X = df.drop(columns='Loan_Status')
y = df['Loan_Status']


# ============================================================================
# --- 7. Spliting Data and Scale Features ---
# ============================================================================

# Split the data into training and testing sets (80% train, 20% test)
# Use stratify=y to ensure the proportion of loan statuses is the same in both sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)

# Scaling numerical features for better performance, especially for Logistic Regression
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)


# ============================================================================
```

```
# --- 8. Train and Evaluate Models ---
# ================================================================================

# --- Model 1: Logistic Regression ---
print("--- Logistic Regression ---")
log_reg = LogisticRegression(random_state=42)
log_reg.fit(X_train_scaled, y_train)
y_pred_log_reg = log_reg.predict(X_test_scaled)

print("Accuracy:", accuracy_score(y_test, y_pred_log_reg))
print("\nClassification Report:\n", classification_report(y_test, y_pred_log_reg))

# --- Model 2: Random Forest Classifier ---
print("\n--- Random Forest Classifier ---")
random_forest = RandomForestClassifier(n_estimators=100, random_state=42)
random_forest.fit(X_train, y_train) # Random Forest doesn't require feature scaling
y_pred_rf = random_forest.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred_rf))
print("\nClassification Report:\n", classification_report(y_test, y_pred_rf))



# ================================================================================
# --- 9. Visualize Model Performance ---
# ================================================================================

# Create confusion matrices to visualize model predictions
cm_log_reg = confusion_matrix(y_test, y_pred_log_reg)
cm_rf = confusion_matrix(y_test, y_pred_rf)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Logistic Regression Confusion Matrix
sns.heatmap(cm_log_reg, annot=True, fmt='d', cmap='Blues', ax=axes[0])
axes[0].set_title('Logistic Regression Confusion Matrix')
axes[0].set_xlabel('Predicted')
axes[0].set_ylabel('Actual')

# Random Forest Confusion Matrix
sns.heatmap(cm_rf, annot=True, fmt='d', cmap='Greens', ax=axes[1])
axes[1].set_title('Random Forest Confusion Matrix')
axes[1].set_xlabel('Predicted')
axes[1].set_ylabel('Actual')

plt.tight_layout()
plt.show()



# ================================================================================
# --- 10. Analyze Feature Importance ---
# ================================================================================
```

```
# Getting feature importances from the trained Random Forest model
importances = random_forest.feature_importances_
feature_names = X.columns
feature_importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': importances})

# Sorting features by importance
feature_importance_df = feature_importance_df.sort_values(by='Importance', ascending=False)

# Ploting the most important features
plt.figure(figsize=(10, 6))
sns.barplot(x='Importance', y='Feature', data=feature_importance_df.head(10))
plt.title('Top 10 Most Important Features (Random Forest)')
plt.xlabel('Importance')
plt.ylabel('Feature')
plt.show()

print("\n--- Top 5 Most Important Features ---")
print(feature_importance_df.head(5))
```